SMART CONTRACT AUDIT REPORT

for

Thorswap Migration

Prepared By: Xiaomi Huang

PeckShield

March 10, 2026

Document Properties

Client	Thorswap
Title	Smart Contract Audit Report
Target	Thorswap
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	March 10, 2026	Xuxian Jiang	Final Release
1.0-rc1	March 8, 2026	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1 Introduction 4
- 1.1 About Thorswap . . . 4
- 1.2 About PeckShield . . . 5
- 1.3 Methodology . . . 5
- 1.4 Disclaimer . . . 7

2 Findings 9
- 2.1 Summary . . . 9
- 2.2 Key Findings . . . 10

3 Detailed Results 11
- 3.1 Inconsistent Unlocked Amount Validation in xMETRO . . . 11
- 3.2 Improved Parameter Validation in RewardDistributor . . . 12
- 3.3 Avoidance of Emitting Zero Share AutoCompounded Events . . . 13
- 3.4 Trust Issue of Admin Keys . . . 15

4 Conclusion 17

References 18

1 Introduction

Given the opportunity to review the design document and related smart contract source code of the Thorswap Migration protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About Thorswap

Thorswap is the leading multi-chain decentralized exchange (DEX) aggregator and the primary interface for THORChain. It allows users to swap native, non-custodial crypto assets across over 25+ major blockchains (such as BTC, ETH, and SOL). This audit covers its migration contract that migrates the THOR token on ETH mainnet to a new ERC-20 METRO token (also on ETH mainnet). Migration is user-initiated and uses a cross-chain bridge message (LayerZero OFT pattern). During migration users choose a 3 or 10-month time-lock. The related staking contract xMETRO pays yield denominated in USDC and offers auto-compounding. The basic information of the audited protocol is as follows:

Table 1.1: Basic Information of The Thorswap Migration Contracts

Item	Description
Client	Thorswap
Website	https://www.thorswap.finance/
Type	EVM Smart Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	March 10, 2026

In the following, we show the Git repository of reviewed files and the commit hash value used in

this audit.

- https://github.com/aidai524/thorswap-migration-contracts.git (a930a52)

And here is the commit ID after all fixes for the issues found in the audit have been checked in:

- https://github.com/aidai524/thorswap-migration-contracts.git (60f196a)

1.2 About PeckShield

PeckShield Inc. [9] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification

Impact \ Likelihood	*High*	*Medium*	*Low*
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [8]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [7], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the design and implementation of the `Thorswap` migration contracts. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	1	■
Low	2	■ ■
Informational	1	■
Total	4	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in Section 3.

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 1 medium-severity vulnerability, 2 low-severity vulnerabilities, and 1 informational recommendation.

Table 2.1: Key Thorswap Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Low	Inconsistent Unlocked Amount Validation in xMETRO	Business Logic	Resolved
PVE-002	Low	Improved Parameter Validation in Reward-Distributor	Coding Practices	Resolved
PVE-003	Informational	Avoidance of Emitting Zero Share Auto-Compounded Events	Coding Practices	Resolved
PVE-004	Medium	Trust on Admin Keys	Security Features	Mitigated

Besides recommending specific countermeasures to mitigate these issues, we also emphasize that it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms need to kick in at the very moment when the contracts are being deployed in mainnet. Please refer to Section 3 for details.

3 Detailed Results

3.1 Inconsistent Unlocked Amount Validation in xMETRO

- ID: PVE-001
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: xMETRO
- Category: Business Logic [6]
- CWE subcategory: CWE-770 [3]

Description

The Thorswap migration and staking functions are implemented in a core xMETRO contract. Each migration request is represented with a data structure ThorLock, which has a lock amount internally denoted as an uint128 type. Accordingly, current implementation validates the locked amount not larger than type(uint128).max. Our analysis shows this requirement is not consistently enforced.

In the following, we show the implementation of a related function, i.e., claimAndStakeUnlockedThor(). As the name indicates, it is used to claim currently-unlocked THORs and further convert into transferable xMETRO as free shares. We notice it properly validates non-zero unlocked amount, but can be further strengthened with require(unlocked <= type(uint128).max, "xMETRO: amount too large");, similar to other routines requestWithdrawUnlockedThor(), requestWithdrawUnlockedYThor(), and requestWithdrawUnlockedContributor(). Note the same improvement can also be applied to another routine, i.e., claimAndStakeUnlockedContributor().

```
    function claimAndStakeUnlockedThor(uint256 maxLocks) external nonReentrant
        whenNotPaused returns (uint256 sharesMinted) {
        uint256 max = maxLocks == 0 ? defaultMaxThorLocks : maxLocks;
        require(max > 0, "xMETRO: zero max");

        uint256 unlocked;
        unlocked += _consumeThorLocks(_thorLocks3m[msg.sender], thorLockCursor3m[msg.
            sender], msg.sender, true, max);
        unlocked += _consumeThorLocks(_thorLocks10m[msg.sender], thorLockCursor10m[msg.
            sender], msg.sender, false, max);
```

```
        require(unlocked > 0, "xMETRO: nothing unlocked");

        // Convert locked shares -> free (transferable) shares. Total shares stay
            unchanged.
        _decreaseLockedShares(msg.sender, unlocked);
        _mintFreeShares(msg.sender, unlocked);

        emit UnlockedClaimedAsShares(msg.sender, UnstakeSource.Thor, unlocked, unlocked)
            ;
        return unlocked;
    }
```

Listing 3.1: `xMETRO::claimAndStakeUnlockedThor()`

Recommendation Improve the above-mentioned routines to further validate the unlock amount is not larger than `type(uint128).max`.

Status This issue has been fixed by this commit: `60f196a`.

3.2 Improved Parameter Validation in RewardDistributor

- ID: PVE-002
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `RewardDistributor`
- Category: Coding Practices [5]
- CWE subcategory: CWE-1126 [1]

Description

The `Thorswap` migration protocol has another core `RewardDistributor` contract that receives revenues in `USDC` from treasury and calls `xMETRO.notifyRevenue(usdcAmount)` to add rewards to the staking pool `xMETRO`. While examining its constructor function, we notice an improvement to validate the given `xMETRO` has the same reward token.

To elaborate, we show below the constructor implementation of the `RewardDistributor` contract. We notice it validates the given `xMETRO` is non-zero. And the validation can be improved by further ensuring the given `xMETRO` shares the same reward token from the input parameters, i.e., `require(rewardToken_ == IxMETRO(xMetro_).rewardToken());`.

```
    constructor(address xMetro_, address rewardToken_, address owner_) Ownable(owner_) {
        require(xMetro_ != address(0), "RewardDistributor: zero xMETRO");
        require(rewardToken_ != address(0), "RewardDistributor: zero rewardToken");
        xMETRO = IxMETRO(xMetro_);
        rewardToken = IERC20(rewardToken_);
    }
```

Listing 3.2: `RewardDistributor::constructor()`

Recommendation Improve the above constructor to ensure the same reward token is used.

Status This issue has been fixed by this commit: `60f196a`.

3.3 Avoidance of Emitting Zero Share AutoCompounded Events

- ID: PVE-003
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: `xMETRO`
- Category: Coding Practices [5]
- CWE subcategory: CWE-1126 [1]

Description

In `Ethereum`, the `event` is an indispensable part of a contract and is mainly used to record a variety of runtime dynamics. In particular, when an `event` is emitted, it stores the arguments passed in transaction logs and these logs are made accessible to external analytics and reporting tools. `Events` can be emitted in a number of scenarios. One particular case is when system-wide parameters or settings are being changed. Another case is when tokens are being minted, transferred, or burned.

In the following, we use the `xMETRO` contract as an example. This contract has public functions that are used to allow for batch auto-compound, i.e., it aggregates a set of users' pending `USDC` rewards, swap them into `METRO`, then mint `xMETRO` pro-rata. It comes to our attention that it emits an event for each user, i.e. `emit AutoCompounded(users[i], pending, share)` (line 428), when the pending amount is not zero. To improve, we can avoid emitting this event if the share amount is zero.

```
376     function autocompoundBatch(address[] calldata users, uint256 minMetroOut, bytes
            calldata swapData)
377         external
378         nonReentrant
379         whenNotPaused
380         returns (uint256 metroOut)
381     {
382         require(autoCompoundOperators[msg.sender], "xMETRO: only autocompound operator")
                ;
383         require(address(swapAdapter) != address(0), "xMETRO: adapter not set");
384         require(users.length > 0, "xMETRO: empty users");

386         uint256 len = users.length;
387         uint256[] memory pendings = new uint256[](len);

389         uint256 totalPending = 0;


392         for (uint256 i = 0; i < len; i++) {
393             address user = users[i];
```

```
            if (user == address(0)) continue;
            if (!autocompoundEnabled[user]) continue;

            (uint256 pending, int256 accumulated) = _pendingAndAccumulated(user);
            if (pending == 0) continue;

            rewardDebt[user] = accumulated;
            pendings[i] = pending;
            totalPending += pending;
        }

        require(totalPending > 0, "xMETRO: no rewards");


        uint256 metroBefore = METRO.balanceOf(address(this));
        rewardToken.forceApprove(address(swapAdapter), totalPending);
        metroOut = swapAdapter.swap(totalPending, minMetroOut, swapData);
        rewardToken.forceApprove(address(swapAdapter), 0);

        uint256 metroAfter = METRO.balanceOf(address(this));
        uint256 received = metroAfter - metroBefore;
        require(received >= minMetroOut, "xMETRO: slippage");


        for (uint256 i = 0; i < len; i++) {
            uint256 pending = pendings[i];
            if (pending == 0) continue;

            uint256 share = (received * pending) / totalPending;

            if (share > 0) {
                _mintFreeShares(users[i], share);
            }

            emit AutoCompounded(users[i], pending, share);
        }


        emit AutocompoundBatchExecuted(msg.sender, totalPending, received);
        return received;
    }
```

Listing 3.3: `xMETRO::autocompoundBatch()`

Recommendation Revisit the above function to emit the AutoCompounded event only when the related share amount is not zero.

Status This issue has been fixed by this commit: `60f196a`.

3.4 Trust Issue of Admin Keys

- ID: PVE-004
- Severity: Medium
- Likelihood: Low
- Impact: Medium
- Target: `Multiple Contracts`
- Category: Security Features [4]
- CWE subcategory: CWE-287 [2]

Description

Within the `Thorswap` migration contracts, there is a special administrative account, i.e., `owner`. This `owner` account plays a critical role in governing and regulating the system-wide operations (e.g., configure various settings, assign roles, and execute privileged operations). It also has the privilege to control or govern the flow of assets within the protocol contracts. In the following, we examine the privileged account and the related privileged accesses in current contracts.

```
    function setMigrationStartTime(uint256 newStartTime) external onlyOwner {
        require(newStartTime >= block.timestamp, "ThorEscrow: bad start time");
        migrationStartTime = newStartTime;
        emit MigrationStartTimeUpdated(newStartTime);
    }

    /// @notice Set THOR bucket deadlines (onlyOwner).
    function setDeadlines(uint256 newDeadline10M, uint256 newDeadline3M) external
        onlyOwner {
        require(newDeadline10M >= block.timestamp && newDeadline3M >= block.timestamp, "
            ThorEscrow: bad deadline");
        deadline10M = newDeadline10M;
        deadline3M = newDeadline3M;
        emit DeadlinesUpdated(newDeadline10M, newDeadline3M);
    }

    /// @notice Set ratios (onlyOwner, 1e18 scaled).
    function setRatios(uint256 newRatio10M, uint256 newRatio3M, uint256 newRatioYThor)
        external onlyOwner {
        require(newRatio10M > 0 && newRatio3M > 0 && newRatioYThor > 0, "ThorEscrow: bad
             ratio");
        require(ratio10M == 0 && ratio3M == 0 && ratioYThor == 0, "ThorEscrow: ratios
            locked");
        ratio10M = newRatio10M;
        ratio3M = newRatio3M;
        ratioYThor = newRatioYThor;
        emit RatiosUpdated(newRatio10M, newRatio3M, newRatioYThor);
    }

    /// @notice Set mint caps (onlyOwner, denominated in minted METRO).
    function setCaps(uint256 newCap10M, uint256 newCap3M) external onlyOwner {
        require(newCap10M > 0 && newCap3M > 0, "ThorEscrow: bad cap");
```

```
        cap10M = newCap10M;
        cap3M = newCap3M;
        emit CapsUpdated(newCap10M, newCap3M);
    }

    /// @notice Set yTHOR mint cap + deadline (onlyOwner; cap denominated in minted METRO).
    function setYThorLimits(uint256 newCapYThor, uint256 newDeadlineYThor) external onlyOwner {
        require(newCapYThor > 0, "ThorEscrow: bad cap");
        require(newDeadlineYThor >= block.timestamp, "ThorEscrow: bad deadline");
        capYThor = newCapYThor;
        deadlineYThor = newDeadlineYThor;
        emit YThorLimitsUpdated(newCapYThor, newDeadlineYThor);
    }

    /// @notice Pause/unpause (onlyOwner).
    function pause() external onlyOwner {
        _pause();
    }

    function unpause() external onlyOwner {
        _unpause();
    }

    /**
     * @notice Rescue tokens sent to this contract by mistake (onlyOwner).
     */
    function rescueTokens(address token, address to, uint256 amount) external onlyOwner {
        require(token != address(thorToken) && token != address(yThorToken), "ThorEscrow: cannot rescue migration tokens");
        require(to != address(0), "ThorEscrow: bad to");
        IERC20(token).safeTransfer(to, amount);
        emit TokensRescued(token, to, amount);
    }
```

Listing 3.4: Example Privileged Operations in `ThorMigrationEscrow`

We understand the need of the privileged functions for proper contract operations, but at the same time the extra power to these privileged accounts may also be a counter-party risk to the contract users. Therefore, we list this concern as an issue here from the audit perspective and highly recommend making these privileges explicit or raising necessary awareness among protocol users.

Recommendation Promptly transfer the privileged account to the intended `DAO`-like governance contract. All changes to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

Status The issue has been mitigated with the use of a multi-sig to act as the owner.

4 Conclusion

In this audit, we have analyzed the design and implementation of the Thorswap migration contracts that migrate the THOR token on ETH mainnet to a new ERC-20 METRO token (also on ETH mainnet). Migration is user-initiated and uses a cross-chain bridge message (LayerZero OFT pattern). During migration users choose a 3 or 10-month time-lock. The related staking contract xMETRO pays yield denominated in USDC and offers auto-compounding. The current code base is clearly organized and those identified issues are promptly confirmed and fixed.

Meanwhile, we need to emphasize that Solidity-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

PeckShield

References

[1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. https://cwe.mitre.org/data/definitions/1126.html.

[2] MITRE. CWE-287: Improper Authentication. https://cwe.mitre.org/data/definitions/287.html.

[3] MITRE. CWE-770: Allocation of Resources Without Limits or Throttling. https://cwe.mitre.org/data/definitions/770.html.

[4] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[5] MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

[6] MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

[7] MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

[8] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[9] PeckShield. PeckShield Inc. https://www.peckshield.com.